

GUÍA TÉCNICA DE ESTUDIO

NEXUS CARS

Arquitectura & Base de Datos

Manual integral sobre la arquitectura Fullstack, API REST en Node.js/Express, conexión a MySQL con XAMPP y Frontend reactivo en Angular 18.

PROYECTO

Sistema Taller Mecánico

STACK TECNOLÓGICO

Angular 18 · Node.js · Express · MySQL

GESTOR DE BASE DE DATOS

XAMPP / phpMyAdmin (Puerto 3306)

FECHA DE ELABORACIÓN

Septiembre 2026

■ 1. Arquitectura General del Sistema

El sistema se construyó siguiendo una arquitectura desacoplada de **3 capas**:

1. Frontend (Cliente)

Desarrollado en **Angular 18 (Standalone Components)**. Maneja la interfaz de usuario, navegación, señales reactivas (`signals`), formularios con validación y almacenamiento de sesión en `localStorage` .

2. Backend (API REST)

Servidor en **Node.js con Express**. Escucha en el puerto `3000` . Gestiona la lógica del negocio, autenticación, conexión a base de datos y expone rutas JSON protegidas por CORS.

3. Base de Datos Relacional

MySQL ejecutándose en **XAMPP** en el puerto `3306` . Almacena la persistencia de usuarios, roles, vehículos, órdenes de trabajo y servicios asociados.

4. Archivo de Entorno (.env)

Almacena de forma segura las variables configurables del servidor (puerto y credenciales de MySQL) para no exponer contraseñas en el código fuente.

■ 2. Pasos para Conectar la Base de Datos con XAMPP

Sigue esta guía paso a paso para configurar y levantar la base de datos en tu entorno local:

1 Iniciar los módulos en XAMPP Control Panel:

Abre el **XAMPP Control Panel** como Administrador y haz clic en el botón **Start** en los módulos **Apache** y **MySQL**. Ambos deben tornarse de color verde.

2 Abrir phpMyAdmin:

Abre tu navegador web e ingresa a: `http://localhost/phpmyadmin` (o haz clic en el botón *Admin* junto a MySQL en XAMPP).

3 Importar el archivo SQL:

En el menú superior de phpMyAdmin, haz clic en la pestaña **Importar** (*Import*). Haz clic en *Seleccionar archivo* y elige el archivo `database/taller_mecanico.sql` ubicado en la raíz del proyecto. Finalmente presiona **Importar** (*Go*).

Este script crea automáticamente la base de datos `taller_mecanico`, las tablas (`users`, `vehicles`, `work_orders`, `work_order_services`) y los usuarios de prueba.

4 Verificar credenciales en el archivo `server/.env`:

Revisa que el archivo `server/.env` contenga los datos por defecto de XAMPP:

```
PORT=3000
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=taller_mecanico
```

5 Probar la conexión con la API:

Inicia el servidor backend ejecutando `npm run server` y accede desde tu navegador o terminal a:

`http://localhost:3000/api/health`. Debe responder `{"ok": true, "message": "API conectada a MySQL"}`.

■ 3. ¿Cómo se construyó y funciona la API REST?

La API REST reside en `server/index.js` y actúa como el **intermediario** entre el cliente Angular y MySQL.

Librerías principales utilizadas:

- **express**: Framework web para crear rutas HTTP (`GET`, `POST`, `PATCH`) y procesar cuerpos JSON con `app.use(express.json())`.
- **cors**: Habilita *Cross-Origin Resource Sharing*, permitiendo que la app de Angular (`http://localhost:4200`) consulte la API (`http://localhost:3000`) sin bloqueos del navegador.
- **mysql2/promise**: Driver de MySQL con soporte de promesas asíncronas (`async/await`). Se utiliza un **Connection Pool** (`mysql.createPool`) que gestiona y reutiliza conexiones eficientemente.

- `dotenv` : Carga las variables definidas en `server/.env` en el objeto global `process.env` .

Catálogo de Endpoints de la API:

Método	Ruta	Descripción
GET	/api/health	Verifica el estado de la API y la conexión con MySQL.
POST	/api/login	Valida email y contraseña contra la tabla <code>users</code> y retorna el usuario con su rol.
GET / POST	/api/users	Obtiene la lista de usuarios o registra un nuevo cliente.
GET / POST	/api/vehicles	Obtiene todos los vehículos o registra uno nuevo asignado a un cliente.
GET / POST	/api/orders	Lista las órdenes de trabajo con sus servicios o crea una nueva orden.
PATCH	/api/orders/:id/status	Actualiza el estado de una orden (<code>recibido</code> , <code>diagnóstico</code> , <code>reparación</code> , <code>listo</code>).
POST	/api/orders/:id/services	Agrega un servicio adicional a una orden de trabajo existente.

■ 4. Estructura y Funcionamiento del Frontend (Angular 18)

1. Sistema de Rutas y Seguridad (Guards)

Ubicado en `src/app/app.routes.ts` y protegido por `src/app/core/guards/role.guard.ts`:

- `/` → **HomeComponent**: Landing page institucional (Navbar, Hero, Servicios, Contacto, Footer).
- `/login` → **LoginComponent**: Formulario de acceso. Si el usuario ya está autenticado, lo redirige al panel según su rol.
- `/admin` → **AdminDashboardComponent**: Panel de administración protegido con `roleGuard('admin')`.
- `/cliente` → **ClientDashboardComponent**: Portal de clientes protegido con `roleGuard('cliente')`.

2. Servicio Centralizado (`WorkshopService`)

El servicio `src/app/core/services/workshop.service.ts` centraliza todas las operaciones:

- **Estado Reactivo con Signals**: Utiliza `currentUser = signal( ... )`, `vehicles = signal( ... )` y `orders = signal( ... )` para que los componentes se actualicen automáticamente cuando cambian los datos.
- **Persistencia de Sesión**: Guarda el usuario en `localStorage ( 'nexus_user' )` al iniciar sesión y lo borra al cerrar sesión (`logout()`). Esto evita que al recargar la página el usuario sea expulsado.

3. Flujo de Autenticación y Redirección

- 1 **Formulario de Login**: El usuario escribe su correo y contraseña en `/login` y presiona "Ingresar".
- 2 **Petición HTTP**: `WorkshopService` envía una petición `POST` a `http://localhost:3000/api/login`.
- 3 **Consulta SQL**: El servidor Express ejecuta `SELECT * FROM users WHERE email = ? AND password = ?`.
- 4 **Guardado de Sesión**: Al recibir la respuesta exitosa, el frontend actualiza el signal `currentUser` y guarda el token/usuario en `localStorage`.
- 5 **Redirección Inteligente**: Si `user.role === 'admin'` se navega a `/admin`. Si es `'cliente'` se navega a `/cliente`.

■ 5. Resumen de Comandos y Usuarios de Prueba

Comandos para Ejecutar el Proyecto:

```
# Terminal 1: Iniciar Servidor Backend (Puerto 3000)
npm run server

# Terminal 2: Iniciar Frontend Angular (Puerto 4200)
npm start
```

Usuarios de Prueba preconfigurados en MySQL:

Rol	Correo Electrónico	Contraseña	Panel de Destino
Administrador	admin@nexuscars.cl	admin123	/admin (Gestión total)
Cliente	cliente@nexuscars.cl	cliente123	/cliente (Garaje y Órdenes)

¡Consejo para el Estudio!

Puedes abrir las herramientas de desarrollador del navegador (F12) en la pestaña **Network (Red)** para ver en tiempo real cómo viajan las peticiones HTTP entre Angular y la API Express en formato JSON.